Title

Game Location Traversal Using DFS and BFS in C++

Problem Statement

Develop a C++ program to represent game locations as vertices and paths as edges. Apply DFS from a starting location, display the exploration order, and count how many locations were visited. Also apply BFS and compare the traversal orders

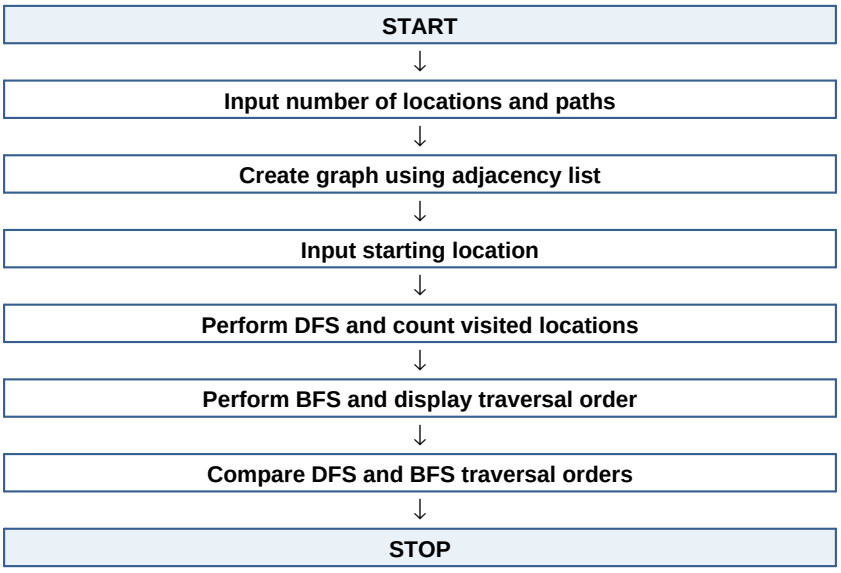
Objectives

1. To represent game locations and paths using a graph.
2. To implement Depth First Search (DFS) from a starting location.
3. To display the DFS exploration order and count the visited locations.
4. To implement Breadth First Search (BFS) traversal.
5. To compare the exploration orders produced by DFS and BFS.

Algorithm

1. Start the program.
2. Read the number of locations and paths.
3. Create an adjacency list to represent the graph.
4. Enter each path as a pair of connected vertices and store both directions.
5. Read the starting location.
6. Run DFS: mark the current vertex as visited, print it, increase the visited count, and recursively visit each unvisited neighbor.
7. Print the DFS traversal order and the number of visited locations.
8. Run BFS using a queue, starting with the selected vertex.
9. Remove the front vertex, print it, and add each unvisited neighbor to the queue.
10. Print the BFS traversal order and compare it with DFS.
11. Stop the program.

Flowchart



C++ Program

```
#include <iostream>
```

```
#include <vector>
#include <queue>
using namespace std;

vector<int> graph[10];
bool visited[10];
int countVisited = 0;

void DFS(int start) {
    visited[start] = true;
    cout << start << " ";
    countVisited++;

    for (int next : graph[start]) {
        if (!visited[next])
            DFS(next);
    }
}

void BFS(int start) {
    bool seen[10] = {false};
    queue<int> q;

    q.push(start);
    seen[start] = true;

    while (!q.empty()) {
        int current = q.front();
        q.pop();
        cout << current << " ";

        for (int next : graph[current]) {
            if (!seen[next]) {
                seen[next] = true;
                q.push(next);
            }
        }
    }
}

int main() {
    int n, e, u, v, start;

    cout << "Enter number of locations: ";
    cin >> n;

    cout << "Enter number of paths: ";
    cin >> e;

    cout << "Enter paths (u v):\n";
    for (int i = 0; i < e; i++) {
        cin >> u >> v;
        graph[u].push_back(v);
        graph[v].push_back(u);
    }

    cout << "Enter starting location: ";
    cin >> start;

    cout << "DFS Traversal: ";
    DFS(start);

    cout << "\nLocations visited: " << countVisited;

    cout << "\nBFS Traversal: ";
    BFS(start);

    cout << "\n";
    return 0;
}
```

Output

```
Enter number of locations: 5
Enter number of paths: 5
Enter paths (u v):
0 1
0 2
1 3
1 4
2 4
Enter starting location: 0
DFS Traversal: 0 1 3 4 2
```

```
Locations visited: 5
BFS Traversal: 0 1 2 3 4
```

Conclusion

The C++ program represents game locations as vertices and paths as edges in a graph. DFS explores locations deeply and counts the visited locations, while BFS explores neighboring locations level by level. The two traversal orders may differ depending on the graph connections and the order in which neighbors are stored. Thus, the program demonstrates two important graph traversal techniques used to explore connected locations in a game map.